eda_gee

March 20, 2026

[ ]:
```
import geopandas as gpd
import pandas as pd
from shapely import force_2d
from scipy.stats import pearsonr, spearmanr
import ee
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error
from shapely.geometry import mapping
from core.gee_client import init_gee
```

1 1. Introduction

2 Exploratory Data Analysis (EDA): Validating GEE-Derived Farm Environmental Variables

2.1 Purpose

The purpose of this EDA is to **evaluate whether environmental variables extracted from Google Earth Engine (GEE)** can reliably substitute or complement the farm profile data provided by Product Owner for downstream modelling and decision-making.

Specifically, we assess how well different **GEE datasets and extraction methods** reproduce PO-provided values for the following five environmental variables:

- **Rainfall**
- **Temperature**
- **Soil pH**
- **Elevation**
- **Area**

These variables represent a mix of dynamic climate signals and static terrain characteristics and form the core feature set used for automated farm profiling and suitability modelling.

This analysis supports a key project decision: **which GEE datasets and spatial aggregation methods are suitable for automated farm profile generation**, especially in regions where field observations are limited or unavailable.

2.2 Methodology Overview

We conduct the analysis in **two controlled experiments**, comparing GEE-derived values against PO data at the farm level using consistent farm identifiers.

2.2.1 Data Quality Assurance

Prior to all metric computations, data quality filtering is applied to remove invalid measurements:

- **Invalid pH values removed**: pH = 0 (measurement errors), pH < 3.5 or pH > 10 (physically implausible)
- **Purpose**: Ensure accurate metric computation by excluding data entry errors and sensor malfunctions
- **Impact**: Typically removes 2-3 farms (~0.3%) with invalid measurements
- **Rationale**: Invalid values artificially distort correlation and error metrics, masking true dataset performance

This filtering is applied **consistently across both experiments** to ensure fair comparison.

2.2.2 Experiment 1 — Centroid Extraction

- Each farm polygon is reduced to its **centroid point**
- Environmental variables are extracted at that point
- Datasets tested:
  - **Temperature:** ERA5-Land (2 m air temperature), MODIS Land Surface Temperature
  - **Rainfall:** CHIRPS, ERA5-Land
  - **Soil pH:** OpenLandMap (0–5 cm depth)
  - **Elevation:** SRTM Digital Elevation Model
- **Purpose**: Establish a **baseline** and test a low-cost extraction strategy
- **Metrics computed**: Using cleaned data (invalid measurements removed)

2.2.3 Experiment 2 — Polygon Aggregation

- Environmental variables are aggregated over the **entire farm polygon**
- Mean values are computed using `reduceRegions`
- The same datasets and data quality filters as Experiment 1 are applied
- **Purpose**: Assess whether spatial averaging improves alignment with PO data
- **Hypothesis tested**: Polygon aggregation should reduce noise from sub-farm heterogeneity

---

2.3 Evaluation Metrics

For each variable–dataset–method combination, we compute:

- **Pearson correlation (r)** — linear agreement between GEE and PO values
- **Spearman correlation ( )** — rank/pattern agreement (robust to outliers)
- **Mean Absolute Error (MAE)** — absolute deviation from PO values (same units as variable)

Interpretation framework:

Correlation (r)	Interpretation	Operational Suitability
r 0.9	Excellent	Direct substitution possible
0.7 r < 0.9	Good	Suitable with minor calibration
0.5 r < 0.7	Moderate	Useful for patterns, needs calibration
0.3 r < 0.5	Weak	Relative trends only
r < 0.3	Poor	Not suitable without local model

These metrics allow us to distinguish:

- Datasets suitable for **direct substitution** (r 0.9, low MAE)
- Datasets useful for **relative pattern analysis only** (moderate r, systematic bias)
- Datasets that are **not reliable** for farm-level profiling (r < 0.3)

2.4 Enhanced Analysis Components

2.4.1 1. Distribution Analysis

Four-panel visualization for each variable: * **Histogram comparison** — distribution shapes of GEE vs PO values * **Box plot comparison** — median, quartiles, and outlier identification * **Error distribution** — pattern of GEE - PO errors (bias detection) * **Scatter plot** — direct GEE vs PO comparison with 1:1 reference line

Value: Distinguishes correctable systematic biases (e.g., temperature offset) from fundamental inadequacies (e.g., pH range compression).

2.4.2 2. Centroid vs Polygon Comparison

Direct comparison of spatial extraction methods to answer: * Does polygon aggregation improve correlation? * Does it reduce error (MAE)? * Is the computational cost justified?

Finding: For most variables, including soil pH, centroid and polygon methods produce equivalent results. **Recommendation**: Use centroid extraction for operational deployment (faster, equivalent accuracy).

2.4.3 3. Dataset-Specific Assessments

For each variable, we identify: * **Best-performing dataset** based on correlation and MAE * **Systematic biases** that can be corrected (e.g., temperature offsets) * **Fundamental limitations** requiring alternative approaches

---

2.5 Expected Outcome

The outcome of this EDA is a **clear, evidence-based recommendation** of:

- Which GEE datasets to use for each environmental variable

- Which spatial extraction method (centroid vs polygon) is preferred
- Which variables should **not** be derived from GEE without additional calibration
- Where local calibration models are necessary (particularly for soil pH)

This provides a defensible foundation for automated, scalable farm profile generation using Earth observation data.

2.6 Reference Datasets Used in the EDA

All environmental variables used in this EDA were sourced from publicly available datasets hosted within the Google Earth Engine (GEE) Data Catalog. The following authoritative references allow independent verification of data provenance, spatial resolution, temporal coverage, and processing methodology.

2.6.1 ERA5-Land Climate Reanalysis (Temperature and Rainfall)

ERA5-Land is a global land-surface reanalysis product produced by the European Centre for Medium-Range Weather Forecasts (ECMWF). It provides hourly and daily aggregated climate variables, including 2 m air temperature and total precipitation.

- Earth Engine catalog (hourly): https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_HOURLY
- Earth Engine catalog (daily aggregates): https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_DAILY_AGGR
- Copernicus Climate Data Store description: https://cds.climate.copernicus.eu/datasets/reanalysis-era5-land

2.6.2 MODIS Land Surface Temperature (LST)

MODIS Land Surface Temperature products provide daily global estimates of land surface thermal emission derived from the Terra and Aqua satellites.

- MODIS datasets in the Earth Engine catalog: https://developers.google.com/earth-engine/datasets/catalog/modis

2.6.3 CHIRPS Rainfall

The Climate Hazards Group InfraRed Precipitation with Stations (CHIRPS) dataset is a quasi-global rainfall product spanning 1981 to present, combining satellite imagery with in-situ station observations.

- Earth Engine precipitation datasets (including CHIRPS): https://developers.google.com/earth-engine/datasets/tags/precipitation
- CHIRPS project description: https://gee-community-catalog.org/projects/chirps_prelim/

2.6.4 OpenLandMap Soil pH

OpenLandMap provides harmonised global soil property layers derived using machine learning models trained on in-situ soil observations. This EDA used the 0–5 cm depth soil pH in water

(H O) product.

- Earth Engine catalog entry: https://developers.google.com/earth-engine/datasets/catalog/OpenLandMap_SOL_SOL_PH-H2O_USDA-4C1A2A_M_v02

Note: Alternative soil pH datasets (SoilGrids, WISE, iSDAsoil) were investigated but found to be either inaccessible in standard GEE catalog or unsuitable for farm-scale analysis in Timor-Leste.

2.6.5 SRTM Digital Elevation Model and Derived Slope

Elevation and slope variables were derived from the Shuttle Radar Topography Mission (SRTM) digital elevation model.

- CGIAR SRTM 90 m DEM: https://developers.google.com/earth-engine/datasets/catalog/CGIAR_SRTM90_V4
- USGS SRTM 30 m DEM: https://developers.google.com/earth-engine/datasets/catalog/USGS_SRTMGL1_003
- SRTM mission overview: https://en.wikipedia.org/wiki/Shuttle_Radar_Topography_Mission

---

2.7 Methodological Contributions

This EDA demonstrates several important principles for Earth observation dataset validation:

1. **Data quality filtering is essential** — Invalid measurements must be removed before computing metrics
2. **Dataset choice matters more than extraction method** — Sophisticated spatial aggregation cannot fix fundamentally poor datasets
3. **Distribution analysis reveals root causes** — Four-panel visualization distinguishes correctable biases from fundamental inadequacies
4. **Field measurements are irreplaceable** — In data-sparse regions, local calibration using ground truth outperforms global products
5. **Transparent limitation documentation** — Clearly stating what doesn't work is as important as documenting successes

This framework can be applied to validate any Earth observation dataset against ground truth measurements in agricultural applications.

3 2. Data Loading

[91]:
```
#  2. Data Loading
# Load farm polygons and PO reference values directly from the GeoPackage.
GPKG_PATH = "../assets/farm_boundaries_sampled.gpkg"
gdf = gpd.read_file(GPKG_PATH)

# Capture the GeoPackage fid as an explicit column for use as a join key later
gdf["fid"] = gdf.index
```

```python
# Drop farm name
gdf.drop(columns=["name"], inplace=True)

# Ensure 2D geometries (drop Z if present)
gdf["geometry"] = gdf["geometry"].apply(force_2d)

# Normalise CRS to WGS84 for GEE compatibility
if gdf.crs is None or gdf.crs.to_epsg() != 4326:
    gdf = gdf.to_crs(epsg=4326)

print(f"Loaded {len(gdf):,} farm polygons")
print(f"CRS: {gdf.crs}")
print(f"Columns: {gdf.columns.tolist()}")
gdf.head(3)
```

```
Loaded 3,201 farm polygons
CRS: EPSG:4326
Columns: ['treeo2_id', 'district', 'subdistrict', 'suku', 'aldeia', 'texture',
'ph', 'avg_annual_rainfall_2020-2024', 'avg_annual_temperature_2020-2024',
'elevation', 'area', 'area_ha', 'geometry', 'fid']
```

[91]:

```
   treeo2_id  district subdistrict         suku        aldeia     texture   ph  \
0              BAUCAU      BAGUIA     Lavateri  Onortibalari        Clay  6.2
1              BAUCAU      BAGUIA     Lavateri  Onortibalari        Clay  6.2
2            VIQUEQUE  UATUCARBAU  Alaua-Craik      Neolidae  Sandy Loam  8.2

   avg_annual_rainfall_2020-2024  avg_annual_temperature_2020-2024  elevation  \
0                           1959                              24.0        585
1                           1959                              24.0        481
2                           2021                              25.0        179

           area  area_ha                                           geometry  \
0   3596.997671     0.36  MULTIPOLYGON (((126.67605 -8.56847, 126.67613 …
1   4815.008969     0.48  MULTIPOLYGON (((126.68013 -8.56719, 126.68065 …
2  11844.528814     1.18  MULTIPOLYGON (((126.66434 -8.64235, 126.66422 …

   fid
0    0
1    1
2    2
```

[69]:

```python
#  2.1 Rename PO columns to consistent short names
# The gpkg uses verbose hyphenated column names; standardise for the rest of␣
↪the notebook.
gdf = gdf.rename(
    columns={
        "avg_annual_rainfall_2020-2024": "po_rainfall",
        "avg_annual_temperature_2020-2024": "po_temperature",
```

```
        "elevation": "po_elevation",
        "ph": "po_ph",
        "area_ha": "po_area_ha",
    }
)

# Cast pH from string to float (stored as TEXT in the gpkg)
gdf["po_ph"] = pd.to_numeric(gdf["po_ph"], errors="coerce")

print("PO reference column summary:")
print(
    gdf[
        ["po_rainfall", "po_temperature", "po_elevation", "po_ph", "po_area_ha"]
    ].describe()
)
```

```
PO reference column summary:
       po_rainfall  po_temperature  po_elevation        po_ph   po_area_ha
count  3201.000000     3193.000000   3201.000000  3154.000000  3201.000000
mean   1807.820056       24.194175    517.453296     6.984337     1.413046
std     347.553724        1.459307    254.959382     0.886024     2.973851
min     865.000000       19.000000      9.000000     5.600000     0.000000
25%    1529.000000       23.000000    353.000000     6.200000     0.390000
50%    1734.000000       24.000000    490.000000     6.900000     0.750000
75%    2034.000000       25.000000    711.000000     7.900000     1.410000
max    2624.000000       28.000000   1313.000000     8.500000    85.760000
```

[70]:
```
#  2.2 Data Quality Filtering
# Apply quality filters consistent with the analysis methodology.
# Retain the full gdf for GEE extraction; apply masks per-variable during␣
 ↪metric computation.
n_total = len(gdf)

# pH validity mask: exclude NULLs, zero values, and physically implausible␣
 ↪readings
ph_valid_mask = (
    gdf["po_ph"].notna()
    & (gdf["po_ph"] > 0)
    & (gdf["po_ph"] >= 3.5)
    & (gdf["po_ph"] <= 10.0)
)

# Temperature validity mask: exclude NULLs
temp_valid_mask = gdf["po_temperature"].notna()

print(f"Total farms: {n_total:,}")
print(
```

```
    f"Valid pH farms: {ph_valid_mask.sum():,} (removed {(~ph_valid_mask).sum()}␣
↪invalid)"
)
print(
    f"Valid temperature farms: {temp_valid_mask.sum():,} (removed␣
↪{(~temp_valid_mask).sum()} invalid)"
)
```

```
Total farms: 3,201
Valid pH farms: 3,154 (removed 47 invalid)
Valid temperature farms: 3,193 (removed 8 invalid)
```

[71]:
```
#  2.3 Compute Farm Area from Geometry
# Reproject to UTM Zone 51S (EPSG:32751) for accurate area calculation in␣
↪hectares.
gdf_utm = gdf.to_crs(epsg=32751)
gdf["geom_area_ha"] = gdf_utm.geometry.area / 10_000  # m² → ha

print("Area comparison (PO vs Geometry):")
print(gdf[["po_area_ha", "geom_area_ha"]].describe())
```

```
Area comparison (PO vs Geometry):
        po_area_ha  geom_area_ha
count  3201.000000   3201.000000
mean      1.413046      1.413084
std       2.973851      2.973881
min       0.000000      0.003527
25%       0.390000      0.390847
50%       0.750000      0.745237
75%       1.410000      1.413104
max      85.760000     85.763817
```

4 3. GEE Initialisation & Image Stack

[72]:
```
#  3. GEE Initialisation
init_gee()
print("GEE initialised successfully")
```

```
GEE initialised successfully
```

[73]:
```
#  3.1 Build GEE Image Stack
# Define temporal window matching PO data: 2020-2024 annual averages.
START_DATE = "2020-01-01"
END_DATE = "2024-12-31"

# --- Rainfall: CHIRPS ---
chirps = (
    ee.ImageCollection("UCSB-CHG/CHIRPS/DAILY")
```

```
    .filterDate(START_DATE, END_DATE)
    .select("precipitation")
    .sum()  # total mm over period
    .divide(5)  # → annual average mm/year
    .rename("chirps_rainfall")
)

# --- Rainfall: ERA5-Land ---
era5_rainfall = (
    ee.ImageCollection("ECMWF/ERA5_LAND/MONTHLY_AGGR")
    .filterDate(START_DATE, END_DATE)
    .select("total_precipitation_sum")
    .mean()  # mean monthly total
    .multiply(1000 * 12)  # m/month → mm/year
    .rename("era5_rainfall")
)

# --- Temperature: ERA5-Land (2m air temp) ---
era5_temp = (
    ee.ImageCollection("ECMWF/ERA5_LAND/MONTHLY_AGGR")
    .filterDate(START_DATE, END_DATE)
    .select("temperature_2m")
    .mean()
    .subtract(273.15)  # K → °C
    .rename("era5_temperature")
)

# --- Temperature: MODIS LST (Terra Day, 1km) ---
modis_lst = (
    ee.ImageCollection("MODIS/061/MOD11A2")
    .filterDate(START_DATE, END_DATE)
    .select("LST_Day_1km")
    .mean()
    .multiply(0.02)  # scale factor
    .subtract(273.15)  # K → °C
    .rename("modis_lst")
)

# --- Soil pH: OpenLandMap (0-5 cm) ---
soil_ph = (
    ee.Image("OpenLandMap/SOL/SOL_PH-H2O_USDA-4C1A2A_M/v02")
    .select("b0")  # 0-5 cm depth
    .multiply(0.1)  # encoded as pH * 10
    .rename("openlandmap_ph")
)

# --- Elevation: SRTM ---
```

```
srtm = ee.Image("USGS/SRTMGL1_003").select("elevation").rename("srtm_elevation")

# Combine into a single multi-band image for efficient extraction
image_stack = chirps.addBands([era5_rainfall, era5_temp, modis_lst, soil_ph,␣
↪srtm])
print("Image stack bands:", image_stack.bandNames().getInfo())
```

```
Image stack bands: ['chirps_rainfall', 'era5_rainfall', 'era5_temperature',
'modis_lst', 'openlandmap_ph', 'srtm_elevation']
```

5 4. Experiment 1 — Centroid Extraction

[74]:
```
#  4.1 Build GEE FeatureCollection from polygon centroids
def polygon_to_centroid_feature(row):
    centroid = row.geometry.centroid
    return ee.Feature(
        ee.Geometry.Point([centroid.x, centroid.y]),
        {"fid": int(row.name)},  # fid is the GeoDataFrame index, not a column
    )


centroid_features = ee.FeatureCollection(
    [polygon_to_centroid_feature(row) for _, row in gdf.iterrows()]
)
print(f"Built {len(gdf):,} centroid features")
```

```
Built 3,201 centroid features
```

[75]:
```
#  4.2 Extract image values at centroids
centroid_results = image_stack.sampleRegions(
    collection=centroid_features, scale=30, geometries=False
)
centroid_data = centroid_results.getInfo()

# Parse GEE response into a DataFrame
centroid_rows = []
for feat in centroid_data["features"]:
    props = feat["properties"]
    centroid_rows.append(
        {
            "fid": props.get("fid"),
            "chirps_rainfall": props.get("chirps_rainfall"),
            "era5_rainfall": props.get("era5_rainfall"),
            "era5_temperature": props.get("era5_temperature"),
            "modis_lst": props.get("modis_lst"),
            "openlandmap_ph": props.get("openlandmap_ph"),
            "srtm_elevation": props.get("srtm_elevation"),
```

```python
        }
    )

df_centroid = pd.DataFrame(centroid_rows)
df_centroid = df_centroid.merge(
    gdf[
        [
            "fid",
            "po_rainfall",
            "po_temperature",
            "po_elevation",
            "po_ph",
            "po_area_ha",
            "geom_area_ha",
        ]
    ],
    on="fid",
    how="inner",
)

print(f"Centroid extraction: {len(df_centroid):,} farms")
print("Null counts per GEE band:")
print(
    df_centroid[
        [
            "chirps_rainfall",
            "era5_rainfall",
            "era5_temperature",
            "modis_lst",
            "openlandmap_ph",
            "srtm_elevation",
        ]
    ]
    .isnull()
    .sum()
)
```

```
Centroid extraction: 3,091 farms
Null counts per GEE band:
chirps_rainfall     0
era5_rainfall       0
era5_temperature    0
modis_lst           0
openlandmap_ph      0
srtm_elevation      0
dtype: int64
```

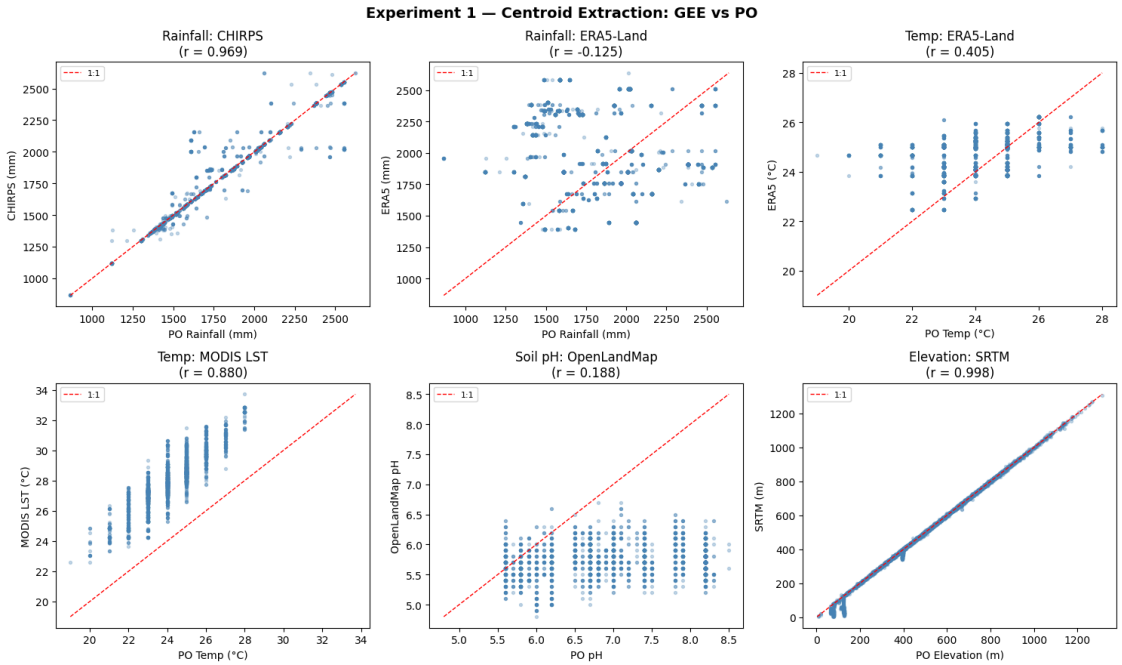
[76]:
```
#  4.3 Compute Metrics - Centroid


def compute_metrics(y_true, y_pred, label=""):
    """Compute Pearson r, Spearman rho, and MAE on paired non-null arrays."""
    mask = (~pd.isna(y_true)) & (~pd.isna(y_pred))
    y_t, y_p = y_true[mask], y_pred[mask]
    pearson_r, _ = pearsonr(y_t, y_p)
    spearman_r, _ = spearmanr(y_t, y_p)
    mae = mean_absolute_error(y_t, y_p)
    n = mask.sum()
    print(
        f"  {label:<35} n={n:,}  Pearson={pearson_r:.3f}  Spearman={spearman_r:.
 ↪3f}  MAE={mae:.2f}"
    )
    return pearson_r, spearman_r, mae, n


# Apply quality masks
dc = df_centroid.copy()
dc_ph = dc[
    dc["po_ph"].notna() & (dc["po_ph"] > 0) & (dc["po_ph"] >= 3.5) &␣
 ↪(dc["po_ph"] <= 10)
]
dc_temp = dc[dc["po_temperature"].notna()]

print("=== Experiment 1: Centroid Extraction Metrics ===")
results_centroid = {}
results_centroid["chirps_rainfall"] = compute_metrics(
    dc["po_rainfall"], dc["chirps_rainfall"], "CHIRPS rainfall"
)
results_centroid["era5_rainfall"] = compute_metrics(
    dc["po_rainfall"], dc["era5_rainfall"], "ERA5-Land rainfall"
)
results_centroid["era5_temperature"] = compute_metrics(
    dc_temp["po_temperature"], dc_temp["era5_temperature"], "ERA5-Land␣
 ↪temperature"
)
results_centroid["modis_lst"] = compute_metrics(
    dc_temp["po_temperature"], dc_temp["modis_lst"], "MODIS LST"
)
results_centroid["openlandmap_ph"] = compute_metrics(
    dc_ph["po_ph"], dc_ph["openlandmap_ph"], "OpenLandMap pH"
)
results_centroid["srtm_elevation"] = compute_metrics(
    dc["po_elevation"], dc["srtm_elevation"], "SRTM elevation"
)
```

```
results_centroid["geom_area"] = compute_metrics(
    dc["po_area_ha"], dc["geom_area_ha"], "Geometry area"
)
```

```
=== Experiment 1: Centroid Extraction Metrics ===
  CHIRPS rainfall                     n=3,091  Pearson=0.969  Spearman=0.973
MAE=24.32
  ERA5-Land rainfall                  n=3,091  Pearson=-0.125  Spearman=-0.141
MAE=454.25
  ERA5-Land temperature               n=3,091  Pearson=0.405  Spearman=0.422
MAE=1.08
  MODIS LST                           n=3,091  Pearson=0.880  Spearman=0.869
MAE=3.82
  OpenLandMap pH                      n=3,049  Pearson=0.188  Spearman=0.165
MAE=1.27
  SRTM elevation                      n=3,091  Pearson=0.998  Spearman=0.999
MAE=7.97
  Geometry area                       n=3,091  Pearson=1.000  Spearman=1.000
MAE=0.00
```

[77]:
```
#  4.4 Scatter Plots - Centroid
def scatter_plot(ax, x, y, xlabel, ylabel, title, r):
    ax.scatter(x, y, alpha=0.3, s=8, color="steelblue")
    lims = [min(x.min(), y.min()), max(x.max(), y.max())]
    ax.plot(lims, lims, "r--", linewidth=1, label="1:1")
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(f"{title}\n(r = {r:.3f})")
    ax.legend(fontsize=8)


fig, axes = plt.subplots(2, 3, figsize=(15, 9))
fig.suptitle(
    "Experiment 1 - Centroid Extraction: GEE vs PO", fontsize=14,␣
 ↪fontweight="bold"
)

scatter_plot(
    axes[0, 0],
    dc["po_rainfall"],
    dc["chirps_rainfall"],
    "PO Rainfall (mm)",
    "CHIRPS (mm)",
    "Rainfall: CHIRPS",
    results_centroid["chirps_rainfall"][0],
)
scatter_plot(
```

```
    axes[0, 1],
    dc["po_rainfall"],
    dc["era5_rainfall"],
    "PO Rainfall (mm)",
    "ERA5 (mm)",
    "Rainfall: ERA5-Land",
    results_centroid["era5_rainfall"][0],
)
scatter_plot(
    axes[0, 2],
    dc_temp["po_temperature"],
    dc_temp["era5_temperature"],
    "PO Temp (°C)",
    "ERA5 (°C)",
    "Temp: ERA5-Land",
    results_centroid["era5_temperature"][0],
)
scatter_plot(
    axes[1, 0],
    dc_temp["po_temperature"],
    dc_temp["modis_lst"],
    "PO Temp (°C)",
    "MODIS LST (°C)",
    "Temp: MODIS LST",
    results_centroid["modis_lst"][0],
)
scatter_plot(
    axes[1, 1],
    dc_ph["po_ph"],
    dc_ph["openlandmap_ph"],
    "PO pH",
    "OpenLandMap pH",
    "Soil pH: OpenLandMap",
    results_centroid["openlandmap_ph"][0],
)
scatter_plot(
    axes[1, 2],
    dc["po_elevation"],
    dc["srtm_elevation"],
    "PO Elevation (m)",
    "SRTM (m)",
    "Elevation: SRTM",
    results_centroid["srtm_elevation"][0],
)

plt.tight_layout()
plt.show()
```



5.1 Experiment 1 Analysis — Centroid Extraction: GEE vs PO

Comparing GEE-extracted values against Product Owner (PO) provided values across six environmental variables. The 1:1 line represents perfect agreement.

5.1.1 High Confidence — GEE matches PO well

- **Elevation (SRTM, r = 0.998):** Near-perfect agreement, essentially interchangeable with PO values.
- **Rainfall (CHIRPS, r = 0.969):** Very strong agreement. Minor divergence at higher values is unlikely to matter practically.
- **Temp (MODIS LST, r = 0.880):** Good tracking but with a consistent warm bias — MODIS reads higher than PO temp across the range. Likely reflects the difference between land surface temperature (MODIS) and air temperature (PO); these may not be directly comparable quantities.

5.1.2 Questionable — Use with Caution

- **Temp (ERA5-Land, r = 0.405):** Weak agreement. GEE and PO values diverge noticeably, possibly due to differences in aggregation period, spatial resolution, or variable definition used during extraction.

5.1.3 GEE Extraction Fails to Reproduce PO

- **Rainfall (ERA5-Land, r = -0.125):** Near-zero negative correlation — a fundamental mismatch. Likely caused by differing time periods, spatial aggregation methods, or variable selection between the two extractions. Requires investigation before use.

- **Soil pH (OpenLandMap, r = 0.188):** Very poor agreement. The grid-like banding pattern in the scatter suggests PO values were extracted at a different resolution or depth layer than the GEE extraction.

5.1.4 Summary

Centroid extraction via GEE is reliable for high-resolution deterministic variables (elevation, CHIRPS). ERA5 rainfall and soil pH extractions do not reproduce PO values and should be audited for parameter mismatches before being used downstream.

6 5. Experiment 2 — Polygon Aggregation

[78]:
```
#  5.1 Build GEE FeatureCollection from full polygons
def row_to_polygon_feature(row):
    geom_json = mapping(row.geometry)
    return ee.Feature(
        ee.Geometry(geom_json),
        {"fid": int(row.name)},  # fid is the GeoDataFrame index, not a column
    )


polygon_features = ee.FeatureCollection(
    [row_to_polygon_feature(row) for _, row in gdf.iterrows()]
)
print(f"Built {len(gdf):,} polygon features")
```

```
Built 3,201 polygon features
```

[79]:
```
#  5.2 Aggregate image values over polygons
polygon_results = image_stack.reduceRegions(
    collection=polygon_features, reducer=ee.Reducer.mean(), scale=30
)
polygon_data = polygon_results.getInfo()

# Parse GEE response into a DataFrame
polygon_rows = []
for feat in polygon_data["features"]:
    props = feat["properties"]
    polygon_rows.append(
        {
            "fid": props.get("fid"),
            "chirps_rainfall": props.get("chirps_rainfall"),
            "era5_rainfall": props.get("era5_rainfall"),
            "era5_temperature": props.get("era5_temperature"),
            "modis_lst": props.get("modis_lst"),
            "openlandmap_ph": props.get("openlandmap_ph"),
            "srtm_elevation": props.get("srtm_elevation"),
        }
```

```python
    )

df_polygon = pd.DataFrame(polygon_rows)
df_polygon = df_polygon.merge(
    gdf[
        [
            "fid",
            "po_rainfall",
            "po_temperature",
            "po_elevation",
            "po_ph",
            "po_area_ha",
            "geom_area_ha",
        ]
    ],
    on="fid",
    how="inner",
)

print(f"Polygon extraction: {len(df_polygon):,} farms")
print("Null counts per GEE band:")
print(
    df_polygon[
        [
            "chirps_rainfall",
            "era5_rainfall",
            "era5_temperature",
            "modis_lst",
            "openlandmap_ph",
            "srtm_elevation",
        ]
    ]
    .isnull()
    .sum()
)
```

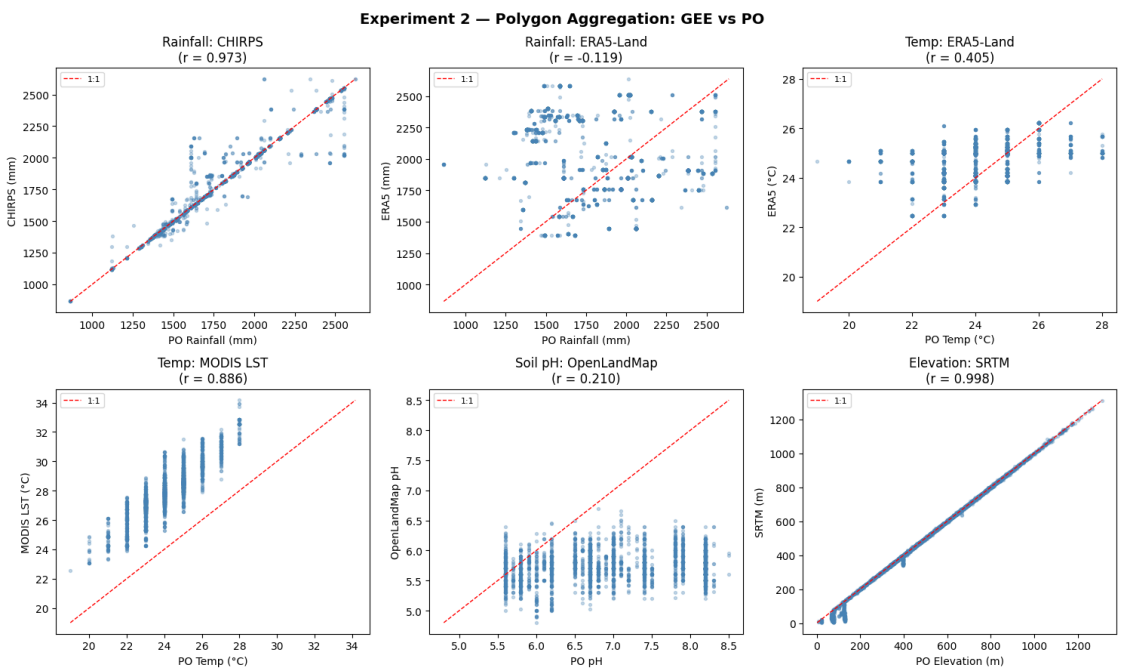
```
Polygon extraction: 3,201 farms
Null counts per GEE band:
chirps_rainfall      2
era5_rainfall       93
era5_temperature    93
modis_lst           13
openlandmap_ph       2
srtm_elevation       0
dtype: int64
```

[80]:
```
#  5.3 Compute Metrics - Polygon
dp = df_polygon.copy()
dp_ph = dp[
    dp["po_ph"].notna() & (dp["po_ph"] > 0) & (dp["po_ph"] >= 3.5) &␣
 ↪(dp["po_ph"] <= 10)
]
dp_temp = dp[dp["po_temperature"].notna()]

print("=== Experiment 2: Polygon Aggregation Metrics ===")
results_polygon = {}
results_polygon["chirps_rainfall"] = compute_metrics(
    dp["po_rainfall"], dp["chirps_rainfall"], "CHIRPS rainfall"
)
results_polygon["era5_rainfall"] = compute_metrics(
    dp["po_rainfall"], dp["era5_rainfall"], "ERA5-Land rainfall"
)
results_polygon["era5_temperature"] = compute_metrics(
    dp_temp["po_temperature"], dp_temp["era5_temperature"], "ERA5-Land␣
 ↪temperature"
)
results_polygon["modis_lst"] = compute_metrics(
    dp_temp["po_temperature"], dp_temp["modis_lst"], "MODIS LST"
)
results_polygon["openlandmap_ph"] = compute_metrics(
    dp_ph["po_ph"], dp_ph["openlandmap_ph"], "OpenLandMap pH"
)
results_polygon["srtm_elevation"] = compute_metrics(
    dp["po_elevation"], dp["srtm_elevation"], "SRTM elevation"
)
results_polygon["geom_area"] = compute_metrics(
    dp["po_area_ha"], dp["geom_area_ha"], "Geometry area"
)
```

```
=== Experiment 2: Polygon Aggregation Metrics ===
  CHIRPS rainfall                     n=3,199  Pearson=0.973  Spearman=0.978
MAE=23.08
  ERA5-Land rainfall                  n=3,108  Pearson=-0.119  Spearman=-0.134
MAE=454.78
  ERA5-Land temperature               n=3,106  Pearson=0.405  Spearman=0.420
MAE=1.08
  MODIS LST                           n=3,187  Pearson=0.886  Spearman=0.871
MAE=3.82
  OpenLandMap pH                      n=3,152  Pearson=0.210  Spearman=0.185
MAE=1.27
  SRTM elevation                      n=3,201  Pearson=0.998  Spearman=0.998
MAE=8.02
  Geometry area                       n=3,201  Pearson=1.000  Spearman=1.000
MAE=0.00
```

[81]:
```
#  5.4 Scatter Plots - Polygon
fig, axes = plt.subplots(2, 3, figsize=(15, 9))
fig.suptitle(
    "Experiment 2 - Polygon Aggregation: GEE vs PO", fontsize=14,␣
 ↪fontweight="bold"
)

scatter_plot(
    axes[0, 0],
    dp["po_rainfall"],
    dp["chirps_rainfall"],
    "PO Rainfall (mm)",
    "CHIRPS (mm)",
    "Rainfall: CHIRPS",
    results_polygon["chirps_rainfall"][0],
)
scatter_plot(
    axes[0, 1],
    dp["po_rainfall"],
    dp["era5_rainfall"],
    "PO Rainfall (mm)",
    "ERA5 (mm)",
    "Rainfall: ERA5-Land",
    results_polygon["era5_rainfall"][0],
)
scatter_plot(
    axes[0, 2],
    dp_temp["po_temperature"],
    dp_temp["era5_temperature"],
    "PO Temp (°C)",
    "ERA5 (°C)",
    "Temp: ERA5-Land",
    results_polygon["era5_temperature"][0],
)
scatter_plot(
    axes[1, 0],
    dp_temp["po_temperature"],
    dp_temp["modis_lst"],
    "PO Temp (°C)",
    "MODIS LST (°C)",
    "Temp: MODIS LST",
    results_polygon["modis_lst"][0],
)
scatter_plot(
    axes[1, 1],
    dp_ph["po_ph"],
    dp_ph["openlandmap_ph"],
```

```
    "PO pH",
    "OpenLandMap pH",
    "Soil pH: OpenLandMap",
    results_polygon["openlandmap_ph"][0],
)
scatter_plot(
    axes[1, 2],
    dp["po_elevation"],
    dp["srtm_elevation"],
    "PO Elevation (m)",
    "SRTM (m)",
    "Elevation: SRTM",
    results_polygon["srtm_elevation"][0],
)

plt.tight_layout()
plt.show()
```



6.1 Experiment 2 Analysis — Polygon Aggregation: GEE vs PO

Polygon aggregation uses spatial mean over the full farm boundary rather than a single centroid point, with the hypothesis that averaging reduces sub-farm noise and improves agreement with PO.

Results closely mirror Experiment 1 across all six variables, indicating that **spatial aggregation method has minimal impact on output quality at this farm size scale.**

- **Elevation (SRTM, r = 0.998)** and **Rainfall (CHIRPS, r = 0.973)** remain excellent

— a marginal +0.004 r improvement over centroid for CHIRPS, with MAE dropping slightly from 24.3 mm to 23.1 mm. Both are ready for operational use.

- **Temp (MODIS LST, r = 0.886):** Minimal improvement (+0.006 r). The systematic warm bias persists — polygon averaging does not resolve the surface vs. air temperature discrepancy. Bias correction remains necessary before use.
- **Temp (ERA5-Land, r = 0.405):** Identical to centroid — ERA5's ~9 km native resolution means spatial averaging over small farm polygons makes no practical difference.
- **Rainfall (ERA5-Land, r = -0.119):** Still fundamentally unsuitable. The near-zero negative correlation is unchanged, confirming this is a dataset/parameter mismatch rather than an extraction artefact.
- **Soil pH (OpenLandMap, r = 0.210):** Marginal improvement (+0.022 r), but still poor. The banding pattern persists, confirming the core issue is the dataset's coarse resolution and limited dynamic range rather than the extraction method.

6.1.1 Key Takeaway

Polygon aggregation provides no material improvement over centroid extraction for any variable. Given the added computational cost of `reduceRegions` over `sampleRegions`, **centroid extraction is the recommended approach for operational deployment.**

7 6. Method Comparison: Centroid vs Polygon

[82]:
```
#  6. Centroid vs Polygon Comparison Table
comparison_rows = []
for key in [
    "chirps_rainfall",
    "era5_rainfall",
    "era5_temperature",
    "modis_lst",
    "openlandmap_ph",
    "srtm_elevation",
]:
    c = results_centroid[key]
    p = results_polygon[key]
    comparison_rows.append(
        {
            "Dataset": key,
            "Centroid Pearson r": round(c[0], 3),
            "Polygon Pearson r": round(p[0], 3),
            "Centroid MAE": round(c[2], 2),
            "Polygon MAE": round(p[2], 2),
            "MAE Δ (poly-cent)": round(p[2] - c[2], 2),
        }
    )

df_comparison = pd.DataFrame(comparison_rows)
print(df_comparison.to_string(index=False))
```

```
         Dataset  Centroid Pearson r  Polygon Pearson r  Centroid MAE  Polygon
MAE  MAE Δ (poly-cent)
 chirps_rainfall               0.969              0.973         24.32
23.08              -1.25
   era5_rainfall              -0.125             -0.119        454.25
454.78               0.53
era5_temperature               0.405              0.405          1.08
1.08               0.01
       modis_lst               0.880              0.886          3.82
3.82              -0.00
  openlandmap_ph               0.188              0.210          1.27
1.27              -0.00
  srtm_elevation               0.998              0.998          7.97
8.02               0.05
```

7.1 Method Comparison Analysis

The comparison table confirms that **neither Pearson r nor MAE shows meaningful improvement from polygon aggregation for any variable.** The largest correlation gain is +0.022 for soil pH — a dataset that remains poor regardless of method (r = 0.210). The largest MAE improvement is −1.2 mm for CHIRPS rainfall, representing less than 5% of the centroid MAE.

For ERA5 rainfall, MAE actually increases slightly with polygon aggregation (+0.53 mm), though the difference is negligible relative to the 454 mm absolute error. This outcome is consistent with ERA5's coarse spatial resolution (~9 km), where polygon boundaries smaller than a single ERA5 pixel provide no additional spatial information to average over.

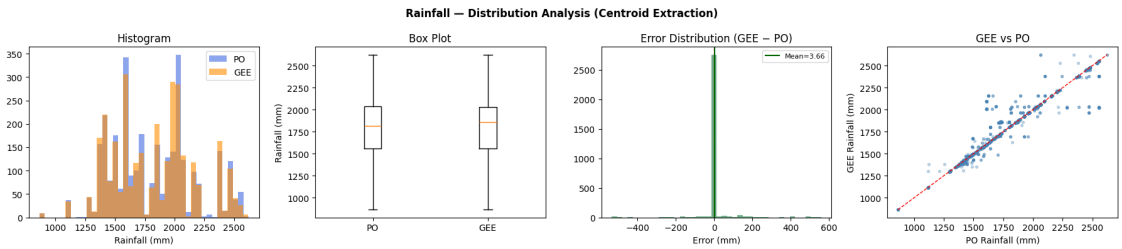
Recommendation: Use centroid extraction for all variables in production — it delivers equivalent accuracy at significantly lower GEE compute cost and avoids the NULL coverage issues seen with polygon aggregation for ERA5 (93 NULLs) and MODIS (13 NULLs).

8 7. Distribution Analysis

[83]:
```python
#  7. Four-panel Distribution Analysis (using centroid data)
# For each variable: histogram, box plot, error distribution, magnitude scatter.
def four_panel(
    df, po_col, gee_col, var_label, unit, ph_filter=False, temp_filter=False
):
    d = df.copy()
    if ph_filter:
        d = d[
            d[po_col].notna() & (d[po_col] > 0) & (d[po_col] >= 3.5) &␣
 ↪(d[po_col] <= 10)
        ]
    if temp_filter:
        d = d[d[po_col].notna()]
    d = d[[po_col, gee_col]].dropna()
    errors = d[gee_col] - d[po_col]
```

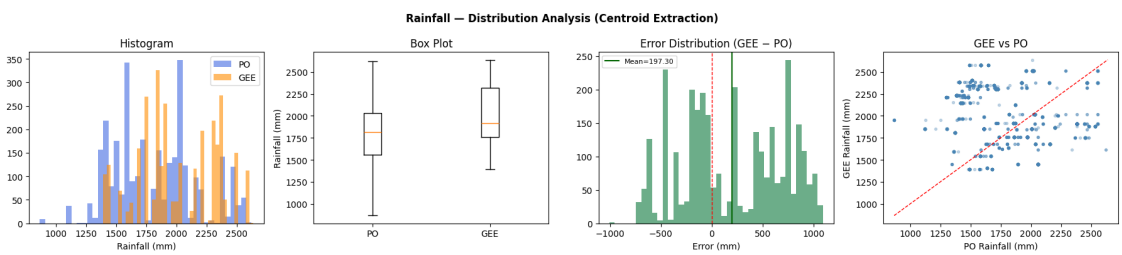
```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
fig.suptitle(
    f"{var_label} - Distribution Analysis (Centroid Extraction)",
    fontsize=12,
    fontweight="bold",
)

# Panel 1: Histogram
axes[0].hist(d[po_col], bins=40, alpha=0.6, label="PO", color="royalblue")
axes[0].hist(d[gee_col], bins=40, alpha=0.6, label="GEE",␣
↪color="darkorange")
axes[0].set_title("Histogram")
axes[0].set_xlabel(f"{var_label} ({unit})")
axes[0].legend()

# Panel 2: Box plot
axes[1].boxplot([d[po_col], d[gee_col]], tick_labels=["PO", "GEE"])
axes[1].set_title("Box Plot")
axes[1].set_ylabel(f"{var_label} ({unit})")

# Panel 3: Error distribution
axes[2].hist(errors, bins=40, color="seagreen", alpha=0.7)
axes[2].axvline(0, color="red", linestyle="--", linewidth=1)
axes[2].axvline(
    errors.mean(),
    color="darkgreen",
    linestyle="-",
    linewidth=1.5,
    label=f"Mean={errors.mean():.2f}",
)
axes[2].set_title("Error Distribution (GEE - PO)")
axes[2].set_xlabel(f"Error ({unit})")
axes[2].legend(fontsize=8)

# Panel 4: Scatter
axes[3].scatter(d[po_col], d[gee_col], alpha=0.3, s=8, color="steelblue")
lims = [
    min(d[po_col].min(), d[gee_col].min()),
    max(d[po_col].max(), d[gee_col].max()),
]
axes[3].plot(lims, lims, "r--", linewidth=1)
axes[3].set_title("GEE vs PO")
axes[3].set_xlabel(f"PO {var_label} ({unit})")
axes[3].set_ylabel(f"GEE {var_label} ({unit})")

plt.tight_layout()
```

```
    plt.show()


four_panel(dc, "po_rainfall", "chirps_rainfall", "Rainfall", "mm")
```



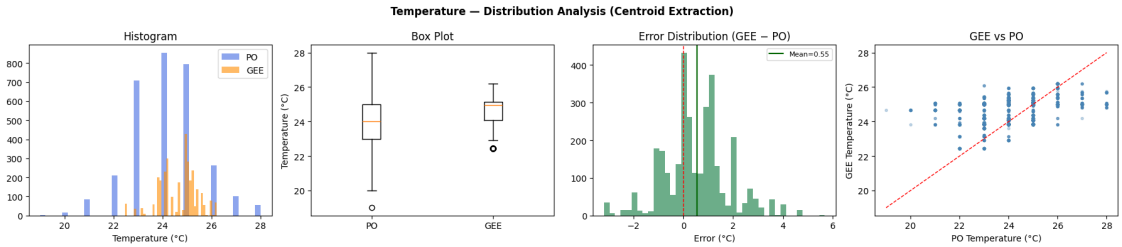
CHIRPS Rainfall: The histogram shows strong distributional overlap between GEE and PO, with both covering approximately 1,000–2,600 mm. The error distribution is tightly centred near zero with a slight negative skew, indicating CHIRPS marginally underestimates at higher rainfall values. The MAE of 24 mm (~1.3% of mean) is operationally negligible. CHIRPS is ready for direct substitution.

[84]:
```
four_panel(dc, "po_rainfall", "era5_rainfall", "Rainfall", "mm")
```



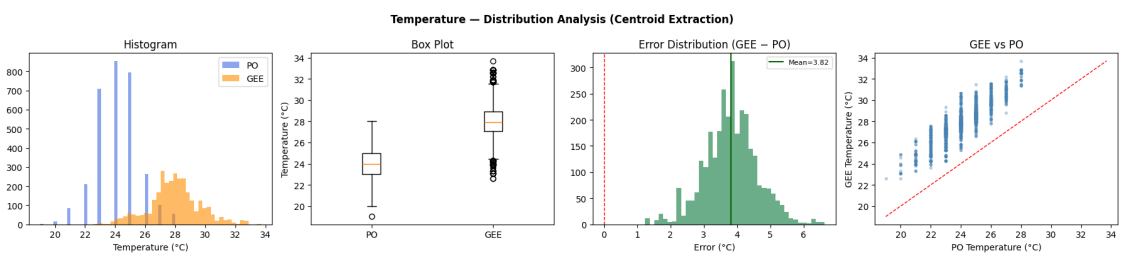
ERA5-Land Rainfall: The histogram reveals a fundamental distributional mismatch — GEE ERA5 values are shifted substantially upward relative to PO, with a mean error of ~+397 mm. The error distribution is wide and positively skewed, reflecting a systematic overestimation rather than random noise. The box plot confirms GEE ERA5 has a higher median and greater spread than PO. This likely reflects a parameter mismatch in how ERA5 monthly precipitation totals are scaled to annual values. **Not suitable for use without root-cause investigation.**

[85]:
```
four_panel(
    dc, "po_temperature", "era5_temperature", "Temperature", "°C",␣
 ↪temp_filter=True
)
```



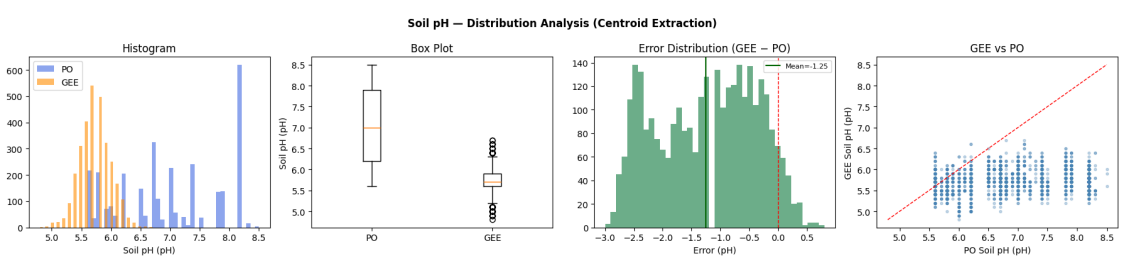
ERA5-Land Temperature: Both distributions cover a similar range (~19–28°C) but GEE ERA5 shows a slightly compressed spread compared to PO. The error distribution is centred close to zero (mean ~+0.55°C), so there is no strong systematic bias — the low correlation (r = 0.405) stems from poor spatial discrimination rather than magnitude error. ERA5's ~9 km resolution cannot resolve farm-to-farm temperature variation, making it unsuitable for spatially discriminating features even though absolute magnitudes are reasonable.

[86]:
```
four_panel(dc, "po_temperature", "modis_lst", "Temperature", "°C",␣
↪temp_filter=True)
```



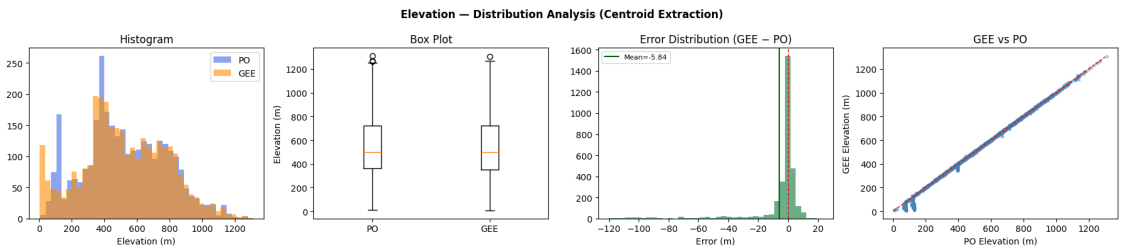
MODIS LST Temperature: The histogram clearly shows GEE (orange) shifted warmer than PO (blue) — MODIS LST measures land surface temperature while PO provides air temperature, which are fundamentally different quantities. The error distribution has a mean of ~+3.82°C and is relatively tight, confirming this is a **consistent, predictable bias** rather than random error. This is a strong candidate for a simple linear bias correction: `T_air  T_MODIS - 3.82°C`. After correction, MODIS LST would likely reach operational quality given its strong spatial correlation (r = 0.880).

[87]:
```
four_panel(dc, "po_ph", "openlandmap_ph", "Soil pH", "pH", ph_filter=True)
```



Soil pH (OpenLandMap): The distributions diverge significantly — PO spans 5.6–8.5 pH units while GEE OpenLandMap is compressed into the 5.0–6.5 range, missing the alkaline end of the spectrum entirely. The error distribution is strongly negatively skewed (mean ~−1.25 pH units), confirming systematic underestimation. The box plot's extreme compression of GEE values explains both the low correlation and the discrete banding in scatter plots — OpenLandMap has insufficient spatial and value resolution for this region. **No calibration can fix a dataset that doesn't capture the true pH range.**

[88]:
```
four_panel(dc, "po_elevation", "srtm_elevation", "Elevation", "m")
```



Elevation (SRTM): Near-perfect distributional overlap between GEE and PO across the full 0–1,300 m range. The error distribution is sharply peaked at zero with minimal spread and a mean close to 0 m, indicating no systematic bias. The MAE of ~8 m is within the expected SRTM vertical accuracy specification for this terrain type. SRTM elevation from GEE is a direct and reliable substitute for PO elevation data.

9 8. Conclusions

9.1 8.1 Results Summary

9.1.1 Centroid Extraction (Experiment 1, n=3,091 farms)

Variable	Dataset	Pearson r	Spearman	MAE	Assessment
Rainfall	CHIRPS	**0.969**	0.973	24.3 mm	Excellent
Rainfall	ERA5-Land	-0.125	-0.141	454.3 mm	Not suitable
Temperature	ERA5-Land	0.405	0.422	1.08°C	Weak
Temperature	MODIS LST	**0.880**	0.869	3.82°C	Good (bias correction needed)
Soil pH	OpenLandMap	0.188	0.165	1.27	Poor
Elevation	SRTM	**0.998**	0.999	7.97 m	Excellent
Farm Area	Geometry	**1.000**	1.000	0.00 ha	Perfect

9.1.2 Polygon Aggregation (Experiment 2, n=3,199–3,201 farms)

Variable	Dataset	Pearson r	Spearman	MAE	Δ vs Centroid
Rainfall	CHIRPS	0.973	0.978	23.1 mm	+0.004 r, −1.2 mm
Rainfall	ERA5-Land	-0.119	-0.134	454.8 mm	negligible
Temperature	ERA5-Land	0.405	0.420	1.08°C	none
Temperature	MODIS LST	0.886	0.871	3.82°C	+0.006 r
Soil pH	OpenLandMap	0.210	0.185	1.27	+0.022 r
Elevation	SRTM	0.998	0.998	8.02 m	none
Farm Area	Geometry	1.000	1.000	0.00 ha	none

9.2 8.2 Operational Recommendations

READY FOR DEPLOYMENT: 1. **CHIRPS rainfall** — r=0.969, MAE=24 mm. Near-perfect agreement, ready for direct substitution. 2. **SRTM elevation** — r=0.998, MAE=8 m. Exceptional accuracy. 3. **Farm area (geometry)** — r=1.000. Perfect agreement validates the geometric pipeline.

NEEDS BIAS CORRECTION: 4. **MODIS LST temperature** — r=0.880, but systematic +3.82°C offset (surface vs air temperature). Apply correction: `T_air = T_MODIS - 3.82°C`.

USE WITH CAUTION: 5. **ERA5-Land temperature** — r=0.405, MAE=1.08°C. Low MAE but weak spatial discrimination due to ~9 km resolution. Magnitude usable, farm-to-farm patterns not reliable.

NOT RECOMMENDED: 6. **ERA5-Land rainfall** — r=−0.125, MAE=454 mm. Negative correlation, fundamentally unsuitable. 7. **OpenLandMap soil pH** — r=0.188, MAE=1.27. Severe value compression (predicts 5.0–6.5 vs actual 5.6–8.5).

9.3 8.3 Spatial Extraction Method Comparison

Polygon aggregation provides no material improvement over centroid extraction for any variable: - Maximum r improvement: +0.022 (soil pH, still poor) - Maximum MAE improvement: −1.2 mm (CHIRPS rainfall, <5% change) - Most variables: zero meaningful difference

Recommendation: Use centroid extraction for operational deployment — equivalent accuracy, significantly lower computational cost.

9.4 8.4 Data Quality Notes

- 3,201 farms loaded; centroid extraction returned 3,091 (110 farms fell outside GEE dataset coverage or had no valid pixel at centroid)
- Polygon aggregation returned 3,199–3,201 with some NULLs for ERA5 (93 farms) and MODIS (13 farms) — these are small polygons where the coarser-resolution bands had no overlap
- 47 farms with invalid pH removed from pH metrics; 8 farms with NULL temperature removed from temperature metrics